

졸업프로젝트 월간 보고서 (7월)

이동혁(2021086917)

최현준(2021037401)

요약

Host-managed zoned block device 위에서 random logical write를 sequential physical write로 변환하는 M1 LSM engine을 구현함.

Underlying device의 실제 zone 정보를 조회하여 zone-aware allocation을 수행하고, logical-to-physical mapping을 active/immutable MemTable과 synchronous compaction으로 관리함.

`docs/07-milestones.md`에 정의된 100 MiB 4 KiB random-write acceptance test를 통과함.

목적

상위 DM device는 conventional block device로 제공하면서, 하위 ZNS device의 sequential-write 제약은 LSM engine 내부에서 처리하고자 함.

Random logical write마다 zone write pointer 기준으로 새로운 physical sector를 할당하고 mapping을 저장함으로써 다음 동작을 제공함.

- Random logical write를 sequential physical write로 변환
- Logical read를 최신 physical 위치로 remap
- 동일 LBA overwrite 시 최신 mapping 반환
- Zone capacity 소진 시 다음 writable zone으로 전환
- MemTable이 커질 경우 freeze 및 compaction 수행

이번 PR은 raw block device 기반 M1 기능을 완성하고, 이후 ext4 연동과 GC를 구현할 기반을 마련하는 것을 목적으로 함.

변경 내용

Underlying zoned device의 실제 zone metadata를 조회하는 기능을 추가함.

조회하는 정보:

- Zone ID
- Start sector
- Zone length
- Zone capacity
- Current write pointer
- Zone condition 및 active 상태

Zone length와 실제 writable capacity를 구분하고, allocator가 zone 시작점이 아닌 underlying device가 보고한 현재 write pointer부터 allocation을 시작하도록 함.

Zone-aware physical allocator를 추가함.

- 4 KiB 단위 sequential allocation

- Zone capacity 경계 검사
- Full, offline, read-only zone 제외
- 현재 zone 소진 시 다음 writable zone으로 전환
- 모든 zone 소진 시 `ENOSPC` 반환
- 동시 allocation 시 physical sector 중복 방지

LSM engine의 write path를 구현함.

```
random logical write
→ zone-aware allocator에서 physical sector 할당
→ MemTable에 최신 mapping 저장
→ underlying physical sector로 bio remap
```

LSM engine의 read path를 구현함.

```
active MemTable lookup
→ immutable MemTable lookup
→ mapping이 존재하면 physical sector로 remap
→ mapping이 없으면 zero-fill
```

동일 logical block이 반복 기록되면 기존 MemTable entry의 physical sector와 sequence를 갱신하여 최신 mapping을 유지함.

MemTable을 active/immutable 세대 구조로 확장함.

```
active MemTable이 threshold 도달
→ 기존 active를 immutable로 전환
→ 새로운 빈 active 설치
```

새 active를 먼저 준비한 후 lock 아래에서 포인터를 교체함. 할당 또는 freeze가 실패하면 기존 active와 mapping을 유지하여 데이터 유실을 방지함.

Active와 immutable을 함께 조회하는 generation lookup을 추가함.

```
active
→ immutable
→ mapping 없음
```

동일 LBA가 두 세대에 존재하면 active의 최신 mapping을 우선함.

Synchronous MemTable compaction을 추가함.

```
immutable A
+ threshold에 도달한 active B
→ merged immutable C
+ 새로운 active D
```

병합 규칙:

- Older에만 존재하는 LBA는 older mapping 유지
- Newer에만 존재하는 LBA는 newer mapping 유지

- 양쪽에 존재하는 LBA는 newer mapping 유지
- 중복 LBA는 결과에서 하나만 유지

Merged immutable과 새 active를 모두 준비한 후 `table_lock` 아래에서 포인터를 원자적으로 교체함.

Compaction이나 allocation이 실패하면 기존 두 테이블과 이미 기록된 mapping을 유지하고, 완료된 write를 I/O error로 변경하지 않음. 이후 write에서 compaction을 다시 시도함.

Compaction은 in-memory RB-tree metadata 병합이며, underlying device에 추가 physical write를 발생시키지 않음.

동시성 및 안정성

Physical allocator는 spinlock으로 보호하여 동시 write에서 동일 physical sector가 중복 할당되지 않도록 함.

Active/immutable 상태는 `table_lock` 으로 보호함.

Lock 순서는 다음과 같이 통일함.

```
table_lock
→ individual MemTable lock
```

Lookup이 끝날 때까지 대상 MemTable의 수명을 보호하고, compaction 시 새 테이블을 완성한 뒤 포인터를 교체함.

실패 시 다음 상태를 보존함.

- 현재 active pointer
- 현재 immutable pointer
- 이미 성공한 logical-to-physical mapping
- Compaction 전 lookup 결과

테스트

MemTable 단위 테스트를 추가함.

- Insert, update, lookup
- 비정렬 LBA 삽입
- 동일 LBA overwrite
- 최신 physical mapping 및 sequence 확인
- Active → immutable freeze
- Threshold 기반 자동 freeze
- Active 우선 generation lookup
- Immutable에만 존재하는 mapping lookup
- 두 MemTable 병합
- 중복 LBA에서 newer mapping 유지
- 반복 synchronous compaction
- Compaction 및 allocation 실패 시 기존 상태 보존

- 동시 write 중 pointer와 mapping 무결성

Zone metadata 및 allocator 테스트를 추가함.

- 실제 `/dev/nullb0` zone count 조회
- Zone start, length, capacity, WP 검증
- Synthetic zone 순차 allocation
- Zone capacity 경계 확인
- Zone 사이 unusable 영역 제외
- Zone 소진 시 다음 zone 전환
- Partial capacity 처리
- Current WP에서 allocation 시작
- 모든 zone 소진 시 `ENOSPC`
- 동시 allocation 시 physical sector 중복 없음

LSM 통합 테스트를 추가함.

- 4 KiB write/read smoke test
- 4 KiB random write
- Underlying zone WP 증가 확인
- CRC read-after-write
- 동일 LBA overwrite 후 최신 데이터 반환
- Immutable에만 남은 mapping 조회
- Compaction 이후 mapping 조회
- Zone boundary 전환
- Kernel I/O error 및 zone write reject 확인

M1 Acceptance 결과

테스트 환경:

```
Underlying device: /dev/nullb0
Device size: 2 GiB
Zone size: 64 MiB
Zone count: 32
Underlying type: host-managed
Upper DM type: conventional
MemTable threshold: 1024
```

실행 명령:

```
sudo bash tests/m1-acceptance.sh
```

테스트 결과:

```

M1 acceptance test: PASS

upper device: conventional
underlying device: host-managed
memtable threshold: 1024
4 KiB random writes: 25,600
official random-write range: 100 MiB
separate CRC range: 32 MiB
read-after-write CRC: PASS
mapping lookup correctness: indirectly verified by CRC
latest overwrite mapping: PASS
underlying zone WP advanced: PASS
zone capacity boundaries: PASS
new kernel I/O errors: 0
zone write rejects: 0

```

Acceptance test는 하나의 module 및 DM target lifecycle에서 다음을 연속으로 검증함.

```

기존 zns-base target 정리
→ LSM engine 빌드
→ underlying zone reset
→ memtable_threshold=1024로 module 적재
→ conventional DM target 생성
→ 100 MiB 4 KiB random write
→ 별도 32 MiB 영역 CRC 검증
→ 동일 LBA A/B/C overwrite 검증
→ zone WP 및 capacity 경계 검증
→ kernel I/O error 및 zone reject 확인
→ target 및 module 정리

```

사용 방법

Zoned null_blk device를 생성함.

```

sudo bash scripts/nullblk-down.sh
sudo bash scripts/nullblk-up.sh

```

LSM engine을 빌드하고 DM target을 생성함.

```

sudo bash scripts/build-run.sh lsm

```

기본 smoke test:

```

sudo env ZNS_ENGINE=lsm bash tests/m1-smoke.sh

```

개별 회귀 테스트:

```

sudo bash tests/memtable-test.sh
sudo bash tests/allocator-test.sh
sudo bash tests/zone-info-test.sh
sudo bash tests/m1-randwrite.sh
sudo bash tests/m1-verify.sh
sudo bash tests/m1-overwrite.sh

```

M1 최종 판정:

```
sudo bash tests/m1-acceptance.sh
```

성공 기준

- 상위 DM device가 conventional로 노출됨
- Underlying device가 host-managed zoned로 유지됨
- Random logical write가 zone WP 기준 physical sequential write로 변환됨
- 100 MiB 4 KiB random write 25,600회 성공
- Read-after-write CRC 일치
- Overwrite 후 최신 mapping 반환
- Underlying zone WP 증가
- Zone capacity 경계 초과 없음
- Kernel I/O error 0건
- Zone write reject 0건
- Freeze 및 synchronous compaction 이후 mapping 유실 없음

이번 PR에서 하지 않는 것

- 1 KiB 또는 2 KiB I/O 지원
- ext4 filesystem round-trip
- `REQ_OP_ZONE_APPEND`
- Zone open/close 명령 관리
- Full zone reset 및 재사용
- Stale physical block 회수
- Garbage collection
- Mapping metadata persistence
- Crash recovery
- SSTable flush
- Background compaction
- 복잡한 level 정책

커밋별 진행 내용

M1 개발 기준선 및 순차 할당 구조 마련

`8bb5fae, 54d8f99` — M1 smoke test 공통 기능 추가

모듈 빌드·적재, Device Mapper target 생성 및 테스트 종료 후 정리를 공통으로 처리하는 `tests/common.sh` 를 추가했다. 이 공통 기능을 사용하는 `tests/m1-smoke.sh` 에서는 4 KiB 데이터를 한 번 쓰고 다시 읽어 원본과 일치하는지 확인하도록 했다.

M1 Smoke 테스트의 경우 다음을 검증하고자 하였다.

```
null_blk → dm-zns-base 모듈 → Device Mapper 장치  
→ 4 KiB 쓰기 → 4 KiB 읽기 → 데이터 비교
```

smoke test 시작 전에 첫 번째 zone을 reset하도록 변경해 같은 테스트를 반복 실행할 수 있게 했다.

e492782 — allocator 기반 append 엔진 도입

지난 달에 `ZNS_ENGINE` 구조를 만든 이유는 한 코드에 여러 사람이 기능을 계속 붙이면 충돌이 잦고, 새로운 구현이 기존 동작까지 망가뜨릴 수 있었기 때문이었다. 따라서 공통 Device Mapper 코드는 그대로 두고 엔진만 교체할 수 있도록 수정하여, 각자 엔진을 독립적으로 개발하면서도 동일한 환경에서 쉽게 비교하고 테스트할 수 있었다.

그리고 기존 append 엔진이 직접 관리하던 다음 기록 위치를 `zns_allocator`로 분리했다. allocator는 4 KiB 블록 단위로 물리 sector를 순차 할당하고, 공간이 부족하면 `-ENOSPC`를 반환한다.

allocator를 사용하는 `append4k-alloc` 엔진을 새로 추가하고 이를 기본 빌드 대상으로 설정했다. 기존 `append4k` 엔진은 비교 및 회귀 검증을 위해 선택 가능한 상태로 유지했다. 이 변경으로 물리 공간 배치 정책을 I/O 엔진과 독립적으로 확장할 수 있는 기반을 마련했다.

이때의 allocator는 zone의 정보를 알지 못하고 sector 0부터 4 Kib의 단위로 선형 할당만 하였다.

- 논리 4 KiB 쓰기
 1. physical append 위치 선택
 2. logical block → physical sector 배열 갱신
 3. bio를 하위 장치의 physical sector로 remap
- 논리 읽기
 1. 배열에서 physical sector 조회
 2. 하위 장치로 remap
 3. 미매핑 LBA는 zero-fill

allocator 동작과 빌드 검증

8f4046b, 03b36fc — allocator 커널 테스트 추가

allocator가 sector를 순서대로 할당하는지, 용량을 모두 사용한 뒤 `-ENOSPC`를 반환하는지 검증하는 커널 테스트 모듈을 추가했다. 두 worker가 동시에 할당하는 상황에서도 같은 physical sector가 중복 반환되지 않는지 확인해 spinlock 기반 동시성 제어를 검증했다.

인메모리 MemTable 구현

f5bd9c9 — RB-tree 기반 MemTable 추가

논리 블록과 물리 위치의 매핑을 bio 처리 경로와 독립적으로 관리하기 위해 Linux 커널 RB-tree 기반 MemTable을 추가했다. 각 엔트리는 logical block, physical sector 및 갱신 순서를 나타내는 sequence를 저장한다.

초기화, 삽입, 조회, 갱신 및 전체 해제 API를 구현했으며, 별도 커널 테스트 모듈에서 비정렬 순서 삽입, 누락된 키 조회, 중복 삽입 거부 및 매핑 갱신을 검증하도록 했다.

- MemTable 생성 시 우려했던 동시성 문제
MemTable은 나중에 bio 경로에서 여러 I/O가 동시에 접근할 수 있다.

따라서 트리 탐색, 삽입, 갱신, 삭제 및 `sequence` 와 `nr_entries` 변경까지 하나의 `spinlock`으로 보호하였다.

- `GFP_KERNEL`

Most of the time `GFP_KERNEL` is what you need. Memory for the kernel data structures, DMAable memory, inode cache, all these and many other allocations types can use `GFP_KERNEL`.

커널에서 동적 메모리 할당 할 때 `kmalloc()` 을 사용하게 되는데 평상시에는 위와 같은 인자를 붙여서 `kmalloc` 을 사용해야 했다.

```
new_entry = kmalloc(sizeof(*new_entry), GFP_KERNEL);
if (!new_entry)
    return -ENOMEM;
```

추가적으로 위의 과정에서 `sleep`에 들어갈 수 있기 때문에 `spinlock`을 잡은 채로 `sleep`하지 않도록 코드 위치를 신경써주어야 했다.

- `EEXIST, -ENODATA` : 중복 키 처리 등 에러 처리

Linux의 표준 `errno` 상수이다.

- `memtable_insert()` : LBA를 삽입 시 이미 데이터가 존재할 경우 `EEXIST`,
- `memtable_update()` : 없는 LBA를 업데이트 시 `ENODATA`

커널 코드에서는 오류를 음수로 반환하는 관례가 있다. 이를 적용 시켰다.

Such a value can be represented as an error-code integer (-Exxx = failure, 0 = success)

- Sequence Tracking

```
new_entry->sequence = memtable->next_sequence++;
```

이후 확장성을 고려하여 엔트리 삽입/매핑을 갱신할 시 단조증가하는 `sequence` 를 부여했다.

ex) LBA 10 → P100, sequence 1 / LBA 10 → P200, sequence 2

이 `insert`, `update`는 테스트를 위해 작성되었고, 위의 구조를 참고하여 `memtable_put()` - `upsert` 메소드로 변환한다.

`a4daf3e` — 동일 LBA의 최신 매핑 유지

동일한 LBA가 반복 기록될 때 호출자가 삽입과 갱신을 구분하지 않아도 되도록 `memtable_put()` 을 추가했다. LBA가 처음 기록되면 새 노드를 삽입하고, 이미 존재하면 `physical sector`와 `sequence`만 갱신한다.

- 기존 함수 2개로 다음과 같이 조합할 수도 있었겠지만

```
ret = memtable_insert;
if (ret == -EEXIST)
    ret = memtable_update;
```

동시성 문제 (Lock을 잡았다 풀기 때문에)로 인하여 아래와 같이 구현된다.


```
// memtable_insert
if (logical_block < entry->logical_block) {
    link = &parent->rb_left;
} else if (logical_block > entry->logical_block) {
    link = &parent->rb_right;
}

// memtable_update
else {
    entry->physical_sector = physical_sector;
}
```

따라서 RB-tree에는 LBA마다 하나의 엔트리만 존재하며 조희 결과는 항상 가장 최근 쓰기의 물리 위치를 가리킨다.

LSM I/O 경로 및 검증 강화

5882117 — LSM 엔진 skeleton 추가

교체 가능한 엔진 구조에 `ZNS_ENGINE=lsm` 으로 선택할 수 있는 LSM 엔진을 추가했다. 초기화 시 allocator와 MemTable을 준비하고, 종료 시 생성의 역순으로 자원을 해제하도록 lifecycle을 구성했다. 이 단계에서는 엔진 선택과 모듈 링크를 먼저 검증하기 위해 실제 bio 요청은 실패 처리했다.

5f1859d — 4 KiB 정렬 I/O 구현

LSM 엔진에 실제 read/write 매핑 경로를 연결했다.

이 때 LSM 엔진의 I/O 경로는 `append4k-alloc` 에서 검증한 bio 처리 방식을 재사용했다.

나머지 동작은 유지하고 배열 기반 주소 매핑만

RB-tree 기반 MemTable의 `put()` 과 `lookup()` 으로 교체했다.

이 시점부터 4 KiB random write가 하위 디바이스의 순차 위치로 변환되고, 논리 주소를 통한 read-after-write가 가능해졌다.

c850277, ba8e803, a4ceae4, a943455 - Random Write 테스트

- 4 KiB random write 테스트 추가

`fio` 를 이용해 LSM 엔진이 4 KiB 단위 임의 쓰기 요청을 처리하는지 확인하는 `tests/ml-randwrite.sh` 를 추가했다. M1의 핵심 workload를 반복 실행할 수 있는 테스트 진입점을 마련했다.

- random write 이후 논리 읽기 검증

쓰기 요청이 성공하는 것만으로는 MemTable 조희가 정확한지 알 수 없으므로 CRC32C 기반 read-after-write 테스트를 추가했다. `fio` 실행 결과와 함께 테스트 구간에서 발생한 커널 I/O 오류 및 ZNS write reject도 검사하도록 했다.

```
--verify=crc32c
--verify_fatal=1
```

CRC32C는 데이터가 쓰기 전과 읽은 후에 동일한지 확인하기 위한 32비트 체크섬 알고리즘이다.
https://fio.readthedocs.io/en/latest/fio_doc.html#cmdoption-arg-verify

이전까지는 I/O 요청이 오류만 검사했다면 이번에는 실제 내용까지 같은지 검증한다.

- 물리 append 및 읽기 검증 강화

테스트 전후의 underlying zone write pointer를 비교해 random write가 실제로 물리 append로 변환됐는지 확인하도록 했다. 또한 데이터 기록과 CRC verify read 단계를 분리해 논리 읽기 요청이 MemTable을 거쳐 올바른 데이터를 반환하는지 더 명확하게 검증했다.

- 반복 overwrite의 최신 매핑 확인

같은 logical block에 A, B, C 패턴을 차례로 기록한 뒤 최종 읽기 결과가 C와 같고 이전 A, B와는 다른지 확인하는 `tests/ml-overwrite.sh`를 추가했다. 이를 통해 하나의 LBA가 여러 물리 위치에 기록돼도 가장 최근 매핑을 반환함을 직접 검증했다.

테스트를 기능별로 세분화해 이후 allocator와 MemTable 구조를 확장하는 과정에서 기존 random-write 변환과 데이터 정합성을 확인하는 회귀 테스트로 활용했다.

4c9a576 — 하위 디바이스의 zone geometry 조회

zone별 시작 sector, length, 실제 기록 가능 capacity, 현재 write pointer, condition 및 active 여부를 메모리에 저장한다.

Q: 왜 이러한 메타데이터를 메모리에 저장하는가?

A: 현재 할당 단위는 4 KiB (8 sector)이다.

1. Allocator는 writable zone의 현재 write pointer부터 8 sector를 연속으로 기록할 공간이 있는지 확인
 2. 공간이 있으면 해당 위치를 반환한 뒤 내부 write pointer를 전진.
 3. 남은 capacity가 부족하면 다음 writable zone으로 이동.
- 하지만 각 zone마다 write pointer와 기록 가능한 capacity가 따로 있기 때문에 메타데이터는 메모리에 저장되어야 한다.
 - `start_sector`: 해당 zone이 시작되는 실제 물리 주소
 - `length`: 다음 zone까지 포함하는 zone의 전체 주소 범위
 - `capacity`: 해당 zone에서 실제 데이터를 기록할 수 있는 범위
 - `write_pointer`: sequential zone에서 다음 쓰기가 허용되는 위치
 - `condition`: empty, open, closed, full, readonly, offline 등의 상태
 - `active`: open 또는 closed 상태처럼 active zone 수에 포함되는 상태인지 표시

`blkdev_report_zones()`

<https://github.com/torvalds/linux/blob/master/block/blk-zoned.c#L261>

```
* Description:
*   Get zone information starting from the zone containing @sector for a
t most
*   @nr_zones, and call @cb for each zone reported by the device.
*   To report all zones in a device starting from @sector, the BLK_ALL_Z
ONES
*   constant can be passed to @nr_zones.
```

```
* Returns the number of zones reported by the device, or a negative error
rno
* value in case of failure.
```

sector 가 포함된 zone부터 최대 nr_zones 개의 정보를 조회하고,
 보고된 zone마다 callback을 한 번씩 호출하는 방식이다.
 성공하면 실제 보고된 zone 수를 반환한다.

이 callback을 사용해서

```
// [zns-zone.c:67]
reported = blkdev_report_zones(bdev, 0, nr_zones,
                               zns_zone_report_cb, &ctx);

// callback function [zns-zone.c:20]
static int zns_zone_report_cb(struct blk_zone *reported,
                              unsigned int index, void *data)
```

- reported : 현재 zone의 metadata
- index : 현재 zone의 순번
- data : 호출자가 전달한 &ctx

```
struct zns_zone_report_ctx *ctx = data;
struct zns_zone *zone = &ctx->table->zones[index];

zone->start_sector = reported->start;
zone->length = reported->len;
zone->capacity = reported->capacity;
zone->write_pointer = reported->wp;
```

위의 Data를 table context로 바꾼 뒤 table을 채우게 된다.

zone report 결과와 메타데이터 유효성, 자원 정리 동작을 확인하는 테스트도 추가했다. 이로써 allocator가 고정된 선형 용량 대신 실제 디바이스의 zone 상태를 사용할 준비가 완료됐다.

zone-aware 할당

acfd851 — zone 경계를 고려한 물리 블록 할당

기존 linear allocator는 zone length와 실제 writable capacity의 차이를 알지 못해 기록 불가능한 영역이나 zone 경계를 넘는 sector를 반환할 수 있었다.

- 이를 해결하기 위해 조회한 zone metadata를 allocator 내부에 복사하고,

```
owned_zones = kmalloc_array(nr_zones, sizeof(*owned_zones), GFP_KERNEL);
if (!owned_zones)
    return -ENOMEM;
memcpy(owned_zones, zones, nr_zones * sizeof(*owned_zones));
```

- 경계를 고려하여 각 zone의 현재 write pointer부터 블록을 할당하도록 확장했다.

```
// zns-allocator.c:105
if (zns_zone_is_writable(zone) &&
    zone->write_pointer <= zone_end &&
    allocator->sectors_per_block <=
        zone_end - zone->write_pointer) {
    *physical_sector = zone->write_pointer;
    zone->write_pointer += allocator->sectors_per_block
}
```

현재 zone의 capacity가 소진되면 full, readonly, offline zone을 건너뛰고 다음 writable zone으로 이동한다. 부분 capacity와 동시 할당 상황에서도 블록이 경계를 넘거나 physical sector가 중복되지 않는지 테스트했다.

추가적으로 active 여부만 계산하여 full, readonly, offline를 구분할 수 없었던 이전 commit과는 달리 원래 condition을 보존하고 allocator 판단에 사용했다.

```
zone->condition = reported->cond; // zns-zone.c:35
```

6af63a6 — LSM 엔진과 zoned allocator 연결

LSM 엔진 초기화 시 하위 디바이스의 zone table을 조회하고, 해당 정보로 zoned allocator를 초기화하도록 변경했다. 이후 write path의 모든 physical sector는 실제 zone capacity와 현재 write pointer를 기준으로 할당된다.

02426b3 — zone 소진 직후 active zone 전환

블록을 하나 할당한 뒤 다음 블록을 기록할 공간이 남지 않았다면 active zone을 즉시 다음 zone으로 전환하도록 했다.

```
// zns-allocator.c:118
zone->write_pointer += allocator->sectors_per_block;
if (allocator->sectors_per_block >
    zone_end - zone->write_pointer)
    allocator->active_zone++;
```

작은 null_blk zone 두 개를 채운 후 세 번째 zone까지 정상 이동하는 tests/zone-boundary-test.sh를 추가해 zone 간 연속 할당을 검증했다.

MemTable 세대 관리

085a2f7 — active/immutable MemTable 상태 도입

단일 MemTable을 직접 소유하던 LSM 상태를 active 및 immutable MemTable 포인터 구조로 변경했다.

기존: 테이블 세대를 전환하기 위해서 내부 RB-tree 상태를 옮겨야 한다.

변경: 기존 active의 내용을 복사하지 않고 포인터 교체만으로 immutable 세대로 전환할 수 있게 된다.

>

```
struct zns_lsm {
    ...
    // before: struct lsm_memtable memtable;
    struct lsm_memtable *active_memtable;
    struct lsm_memtable *immutable_memtable;
    ...
};
```

- 실제로 동적할당까지 진행해서 유연해졌다.

```
struct lsm_memtable *memtable_create(void)
{
    ...
    memtable = kzalloc(sizeof(*memtable), GFP_KERNEL);
    ...
}
```

MemTable 생성·해제 API와 두 포인터를 보호하는 mutex를 추가했다. 기존 read/write는 우선 active만 사용하도록 유지하면서 이후 freeze와 다세대 조회를 위한 상태 기반을 만들었다.

a837620 — active MemTable freeze 구현

새 active MemTable을 미리 할당한 뒤 lock 안에서 기존 active를 immutable로 전환하는 freeze 기능을 구현했다. immutable이 이미 존재하거나 새 테이블 준비가 실패하면 기존 포인터와 데이터를 그대로 유지해 전환 도중 매핑이 유실되지 않도록 했다.

```
spin_lock(table_lock);

if (!*active) {
    ret = -EINVAL;
} else if (*immutable) {
    ret = -EBUSY;
} else {
    *immutable = *active;
    *active = new_active;
}

spin_unlock(table_lock);
```

- `__maybe_unused`

```
static int __maybe_unused zns_lsm_freeze_memtable;
```

- GCC의 attribute이다. 사용되지 않는 함수도 compiler warning을 내지 않는다.

```
#define __maybe_unused __attribute__((unused))
```

c4e3f43 — entry threshold 기반 자동 freeze

실수로 이전 Commit 내용이 이번 Commit시에도 올라갔다.

앞선 커밋에서는 threshold와 active/immutable 상태 구조만 준비했으며, 실제 write path에서 자동 freeze를 수행하는 동작은 이번 커밋에서 연결했다.

>

LSM write path를 `memtable_put_active()`에 연결해 mapping 저장과 threshold 검사를 하나의 mutex로 직렬화했다.

- `memtable_put_active()`
 1. table mutex를 획득한다.
 2. 현재 active MemTable에 mapping을 저장한다.
 - 저장에 실패하면 오류를 반환한다.
 3. active의 고유 엔트리 수를 확인한다.
 - threshold 미만이면 freeze하지 않는다.
 - immutable이 이미 존재하면 freeze하지 않는다.

이 단계에서는 immutable을 처리하는 경로가 없으므로 active가 threshold 이상으로 계속 증가할 수 있다.

4. 새로운 빈 MemTable을 생성한다.
 - 생성에 실패하면 기존 active에 저장한 mapping을 유지하고 write를 성공으로 처리한다.
5. 기존 active를 immutable로 전환한다.
6. 새 MemTable을 active로 설치한다.
7. table mutex를 해제한다.

의 순서로 진행한다.

- table lock을 `spinlock`에서 `mutex`로 변경하게 된 계기
 1. `memtable_put()`과 새 MemTable 생성 과정에서 `GFP_KERNEL` 메모리 할당이 발생해 실행 중 잠들 수 있기 때문이다.

여러 write가 동시에 threshold에 도달해 중복 freeze하거나 mapping을 잘못된 세대에 저장하지 않도록, mapping 저장부터 포인터 전환까지 **동일한 mutex**로 보호하였다.

be8cd87, 8caab0e — immutable mapping 조회 연결

`memtable_lookup_active_immutable()`을 추가해 table mutex로 두 MemTable 포인터의 수명을 보호하면서 active를 먼저 검색하고, 매핑이 없을 때만 immutable을 검색하도록 했다.

```
// lsm-memtable.c:145
ret = memtable_lookup(*active, logical_block, ...);
if (ret == -ENODATA && *immutable)
    ret = memtable_lookup(*immutable, logical_block, ...);
```

active는 **freeze 이후에 기록된** 더 새로운 세대이기 때문에 같은 LBA가 두 세대에 존재한다면 **active의 결과를 최신 값으로** 읽어야 한다.

- 양쪽에 모두 없을 때는 zero-fill한다.

```
// -ENODATA 반환 = REQ_OP_READ 분기에서 zero_fill_bio()

// zns-engine-lsm.c:185
case REQ_OP_READ:
    ret = zns_append4k_alloc_read(m, bio->bi_iter.bi_sector,
                                   bio_sectors(bio), &physical_sector);
    if (ret == -ENODATA) {
        zero_fill_bio(bio);
        bio_endio(bio);
        return DM_MAPIO_SUBMITTED;
    }
}
```

이후 LSM read path를 이 API에 연결해 freeze 이전의 mapping도 계속 읽을 수 있게 했다.

또한 `memtable_threshold` 를 의미가 없는 기본값이었던 16384 에서 module parameter로 노출해 작은 threshold로 freeze 동작을 재현할 수 있게 했고, `immutable-read-test.sh` 에서 freeze 전후 데이터와 최신 overwrite 결과를 검증했다.

MemTable Compaction

9544fb6 — immutable/active 매핑 병합 기능 추가

older immutable과 newer active MemTable을 logical block 순서로 병합하는 compaction API를 구현했다. 중복 LBA는 newer의 매핑을 선택해 최신 쓰기를 보존하고, 결과 테이블에는 LBA별 엔트리가 하나만 남는다.

(Older table copy → newer table `put()`)

병합이 완전히 성공한 경우에만 결과를 반환하고, 실패하면 부분 결과를 해제하되 입력 테이블은 변경하지 않도록 했다. 이를 통해 compaction 실패 시에도 기존 조회 결과가 유지된다.

46cfe3e — write threshold에서 MemTable compaction 수행

immutable이 존재하는 상태에서 active가 다시 threshold에 도달하면 두 테이블을 동기적으로 병합하고, 비어 있는 새 active를 준비한 뒤 lock 안에서 포인터를 원자적으로 교체하도록 했다.

병합이나 메모리 할당이 실패하면 기존 두 세대와 이미 기록된 매핑을 보존하고 다음 write에서 compaction을 재 시도한다.

87afae2 — 최종 M1 acceptance test 추가

기존 테스트가 실행 순서와 남아 있는 Device Mapper target 상태에 영향을 받을 수 있어, 전체 lifecycle을 독립 적으로 관리하는 `tests/m1-acceptance.sh` 를 추가했다. 이 테스트는 다음 항목을 한 번에 판정한다.

- LSM 엔진 빌드 및 `memtable_threshold=1024` 적용
- 상위 DM 장치의 conventional 속성
- 하위 디바이스의 host-managed 속성과 writable capacity
- 100 MiB 구간에 대한 4 KiB random write 25,600회
- 별도 32 MiB 구간의 CRC32C read-after-write

- 동일 LBA에 대한 A → B → C overwrite와 최신 C 매핑
- zone write pointer 전진 및 capacity 경계 준수
- 테스트 구간의 신규 커널 I/O 오류와 zone write reject가 0건인지 확인
- 성공·실패 이후 target, 모듈 및 임시 파일 정리

71b450d, 71e14ea — main 병합

기능 브랜치에 최신 main 을 병합해 엔진 주입 구조와 M1 구현을 통합했다.

현재 범위와 후속 과제

이번 구현은 M1 범위에 맞춰 4 KiB 정렬 I/O와 인메모리 매핑을 지원한다. 매핑 메타데이터 영속화와 crash recovery는 아직 포함하지 않으며, 모든 writable zone을 소진하면 allocator는 -ENOSPC 를 반환한다. invalid block 회수, victim zone 선정, valid data 이동 및 zone reset을 포함하는 GC는 다음 M3 단계의 과제이다.

다음 단계에서는 VM 작업 트리의 src/Makefile 줄바꿈을 LF로 정리하고 공통 object 복구 내용을 71e14ea 가 포함된 브랜치 상태와 일치시킬 필요가 있다. 이후 ext4를 올려 unmount/remount 뒤에도 데이터가 유지되는지 확인하는 M2 라운드트립으로 확장할 예정이다.